

Maschinelles Lernen (ML)
Lineare Modelle und Perzeptronen

Dr.-Ing. Christoph-Alexander Holst

Themen 3. Block

Eine handkonstruierte lineare Methode

Das einfache Perzeptron

Gradienten-basiertes Lernen

Eine handkonstruierte lineare Methode (Nearest-Centroid)

Eine handkonstruierte lineare Methode (Nearest-Centroid)

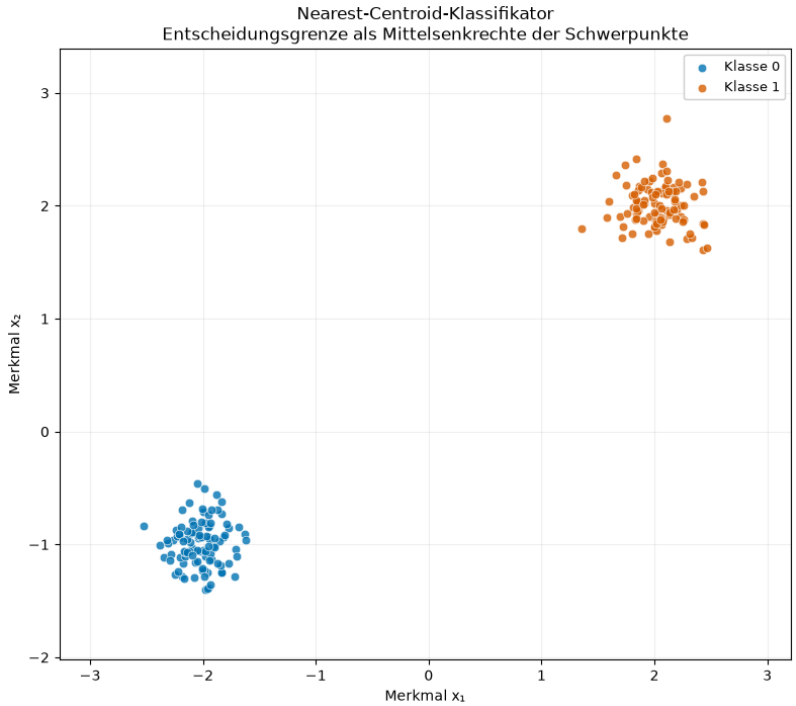
Wir haben:

- Zwei linear trennbare Datencluster
- Annotationen (Label)

Wir suchen:

- Eine diskriminante Gerade (Entscheidungsgrenze):

$$x_2 = m \cdot x_1 + b$$



Eine handkonstruierte lineare Methode (Nearest-Centroid)

Wir haben:

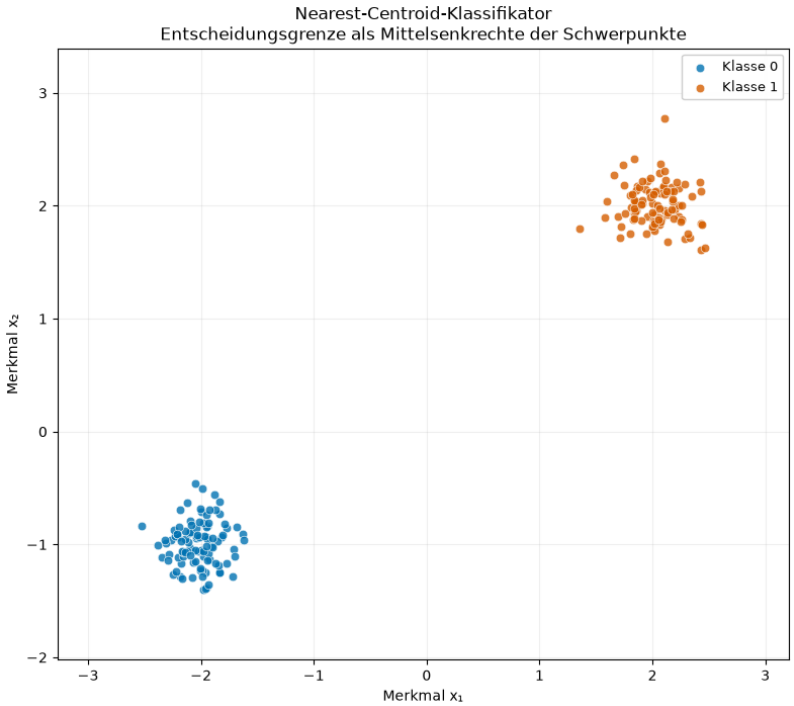
- Zwei linear trennbare Datencluster
- Annotationen (Label)

Wir suchen:

- Eine diskriminante Gerade (Entscheidungsgrenze):

$$x_2 = m \cdot x_1 + b$$

$$w_2 \cdot x_2 = w_1 \cdot x_1 + b$$



Eine handkonstruierte lineare Methode (Nearest-Centroid)

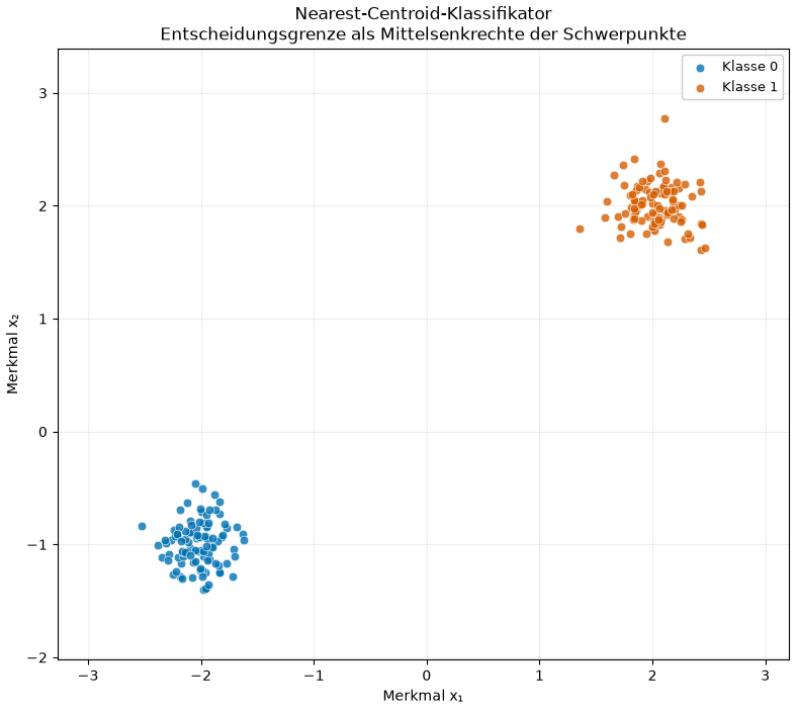
Wir haben:
- Zwei linear trennbare Datencluster
- Annotationen (Label)

Wir suchen:
- Eine diskriminante Gerade (Entscheidungsgrenze):

$$x_2 = m \cdot x_1 + b$$

$$w_2 \cdot x_2 = w_1 \cdot x_1 + b$$

$$w_1 \cdot \mathrm{x}_1 + w_2 \cdot x_2 + b = 0$$



Eine handkonstruierte lineare Methode (Nearest-Centroid)

Wir haben:
- Zwei linear trennbare Datencluster
- Annotationen (Label)

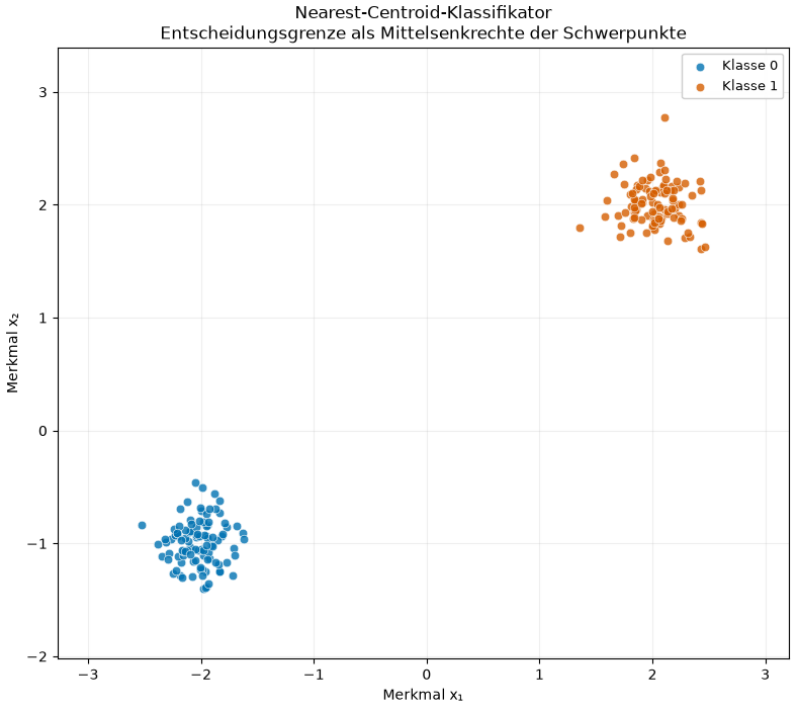
Wir suchen:
- Eine diskriminante Gerade (Entscheidungsgrenze):

$$x_2 = m \cdot x_1 + b$$

$$w_2 \cdot x_2 = w_1 \cdot x_1 + b$$

$$w_1 \cdot \mathrm{x}_1 + w_2 \cdot x_2 + b = 0$$

$$g(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x} + b$$



Eine handkonstruierte lineare Methode (Nearest-Centroid)

Wir haben:

- Zwei linear trennbare Datencluster
- Annotationen (Label)

Wir suchen:

- Eine diskriminante Gerade (Entscheidungsgrenze):

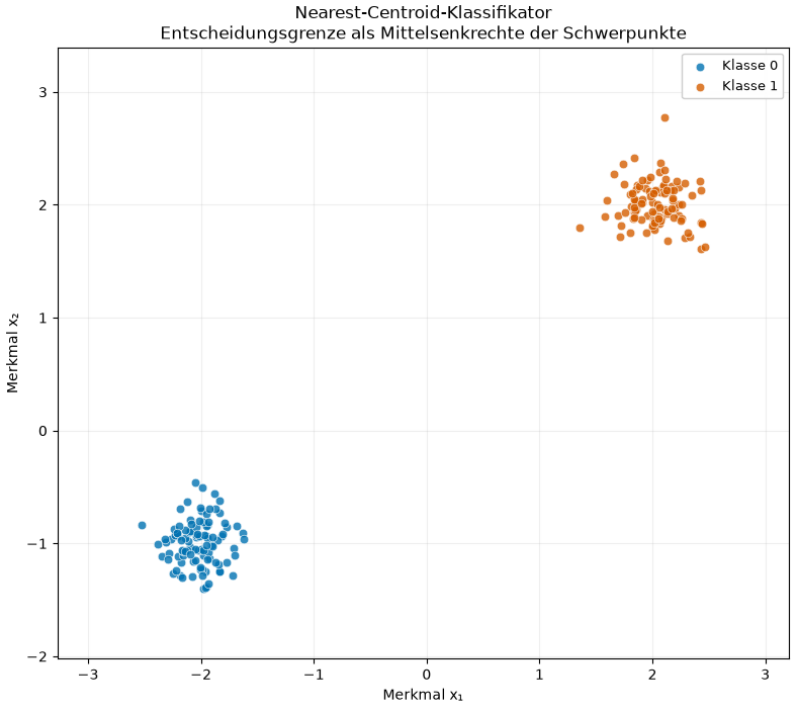
$$x_2 = m \cdot x_1 + b$$

$$w_2 \cdot x_2 = w_1 \cdot x_1 + b$$

$$w_1 \cdot \mathrm{x}_1 + w_2 \cdot x_2 + b = 0$$

$$g(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x} + b$$

$$h(\boldsymbol{x}) = \begin{cases} 0, & g(\boldsymbol{x}) < 0 \\ 1, & g(\boldsymbol{x}) \geq 0 \end{cases}$$



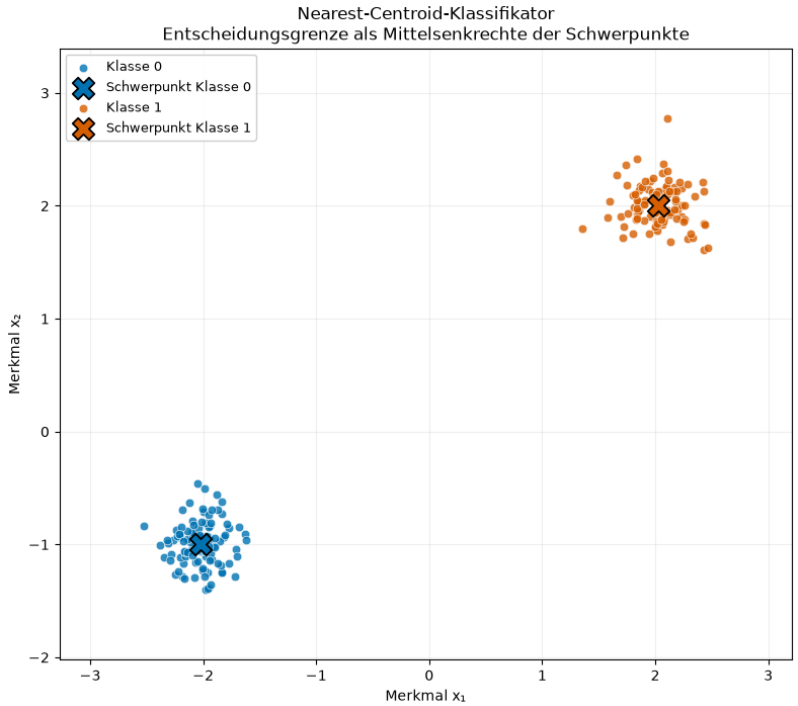
Eine handkonstruierte lineare Methode (Nearest-Centroid)

Idee:

- Schwerpunkte der Cluster über Mittelwerte berechnen
- Gerade zwischen den Schwerpunkten ziehen
- Rechtwinklige Gerade im Mittelpunkt ziehen

Schwerpunkte:

$$\mu_- = \frac{1}{N_-}\sum_i x_{-,i} \quad \mu_+ = \frac{1}{N_+}\sum_i x_{+,i}$$



Eine handkonstruierte lineare Methode (Nearest-Centroid)

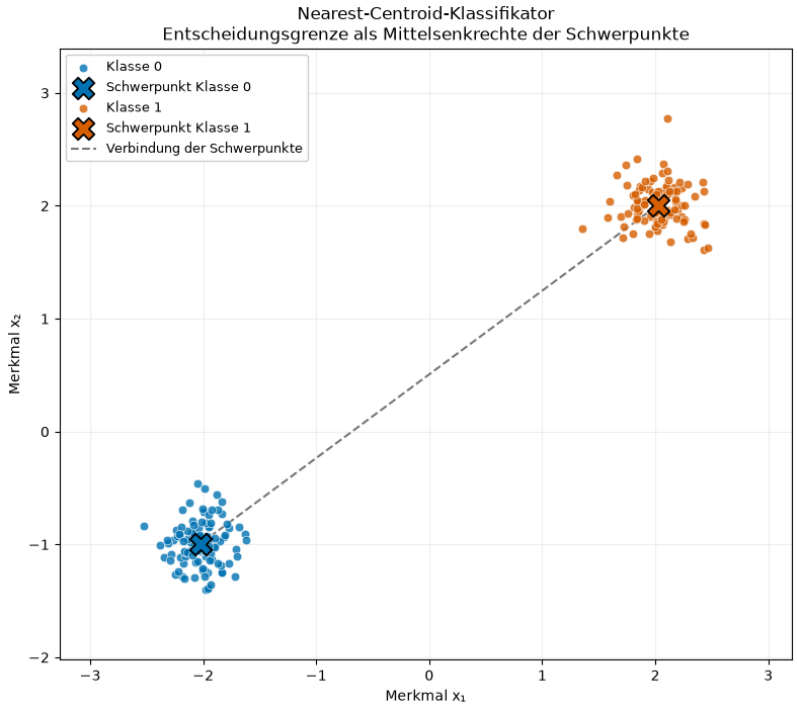
Idee:

- Schwerpunkte der Cluster über Mittelwerte berechnen
- Gerade zwischen den Schwerpunkten ziehen
- Rechtwinklige Gerade im Mittelpunkt ziehen

Verbindung der Schwerpunkte:

$$x_{h,2} = m_h \cdot x_{h,1} + b_h$$

Hilfsgerade



Eine handkonstruierte lineare Methode (Nearest-Centroid)

Idee:

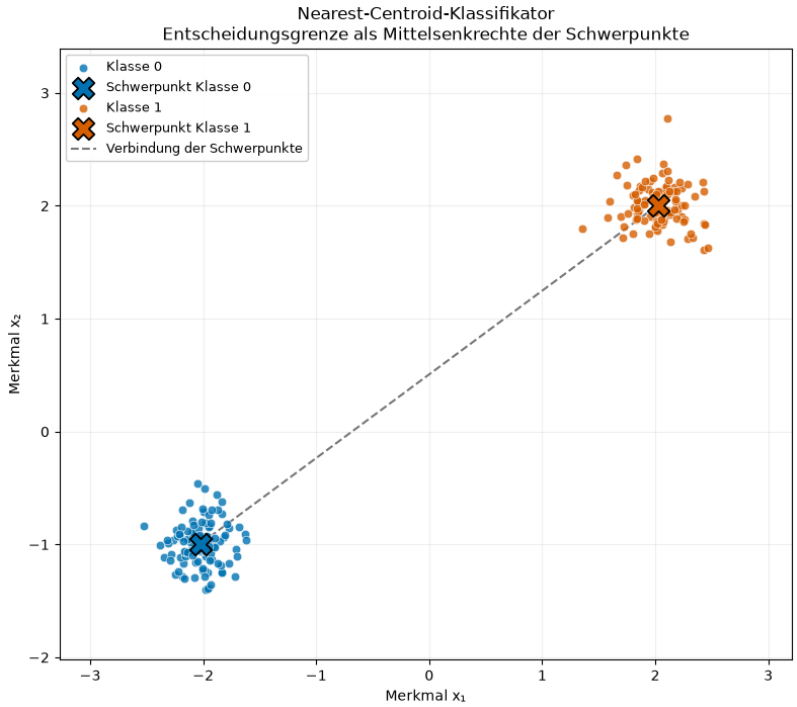
- Schwerpunkte der Cluster über Mittelwerte berechnen
- Gerade zwischen den Schwerpunkten ziehen
- Rechtwinklige Gerade im Mittelpunkt ziehen

Verbindung der Schwerpunkte :

$$x_{h,2} = m_h \cdot x_{h,1} + b_h$$

$$m_h = \frac{\mu_{-,2} - \mu_{+,2}}{\mu_{-,1} - \mu_{+,1}}$$

$$b = \mu_{+,2} - m_h \cdot \mu_{+,1}$$



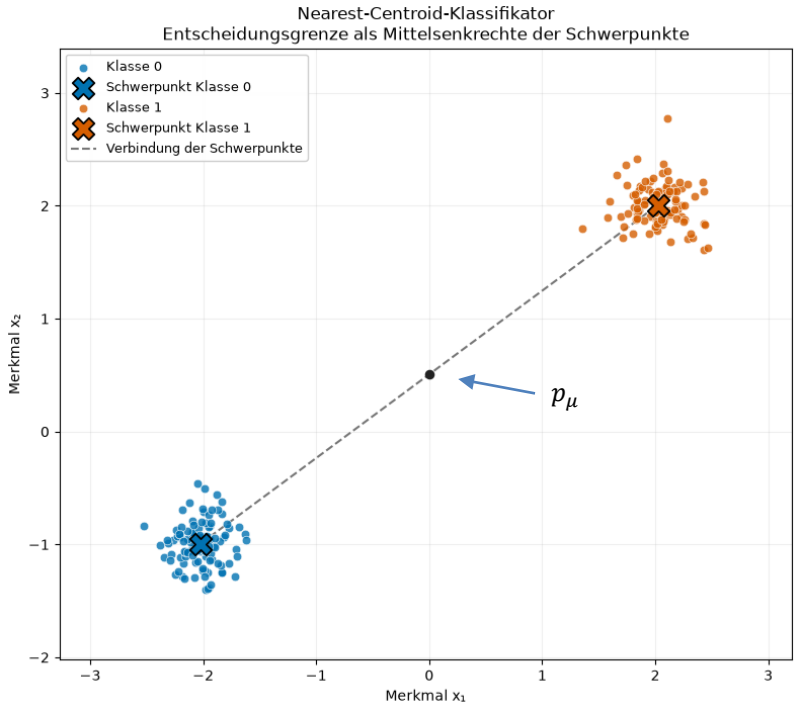
Eine handkonstruierte lineare Methode (Nearest-Centroid)

Idee:

- Schwerpunkte der Cluster über Mittelwerte berechnen
- Gerade zwischen den Schwerpunkten ziehen
- Rechtwinklige Gerade im Mittelpunkt ziehen

Mittelpunkt p_μ der Strecke $\overline{\mu_-, \mu_+}$

$$p_\mu = \left(\frac{\mu_{-,1} + \mu_{+,1}}{2}, \frac{\mu_{-,2} + \mu_{+,2}}{2}\right)$$



Eine handkonstruierte lineare Methode (Nearest-Centroid)

Idee:

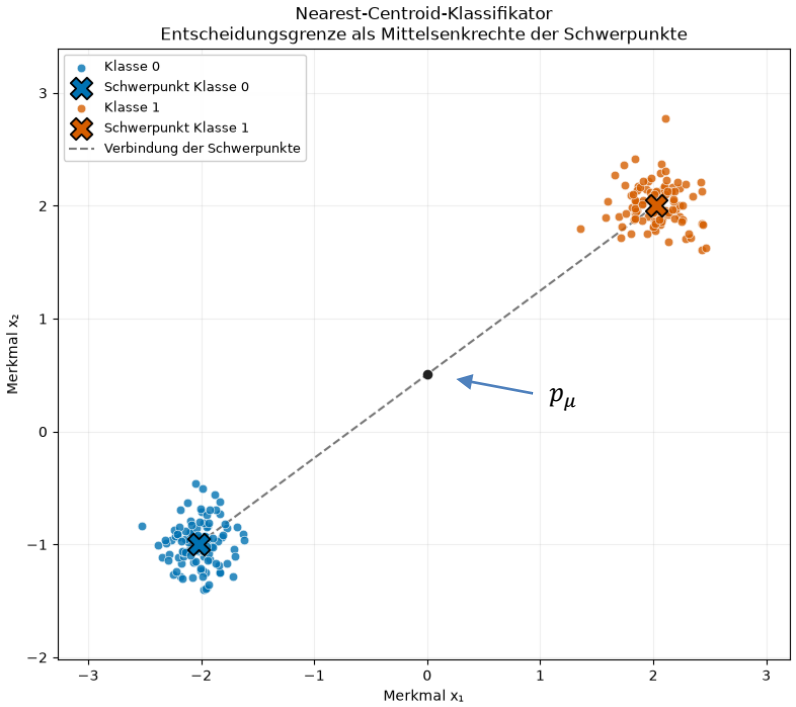
- Schwerpunkte der Cluster über Mittelwerte berechnen
- Gerade zwischen den Schwerpunkten ziehen
- Rechtwinklige Gerade im Mittelpunkt ziehen

Mittelpunkt p_μ der Strecke $\overline{\mu_-, \mu_+}$

$$p_\mu = \left(\frac{\mu_{-,1} + \mu_{+,1}}{2}, \frac{\mu_{-,2} + \mu_{+,2}}{2}\right)$$

Steigungsverhältnis rechtwinkliger Geraden:

$$m = -\frac{1}{m_h}$$



Eine handkonstruierte lineare Methode (Nearest-Centroid)

Idee:

- Schwerpunkte der Cluster über Mittelwerte berechnen
- Gerade zwischen den Schwerpunkten ziehen
- Rechtwinklige Gerade im Mittelpunkt ziehen

Mittelpunkt p_μ der Strecke $\overline{\mu_-, \mu_+}$

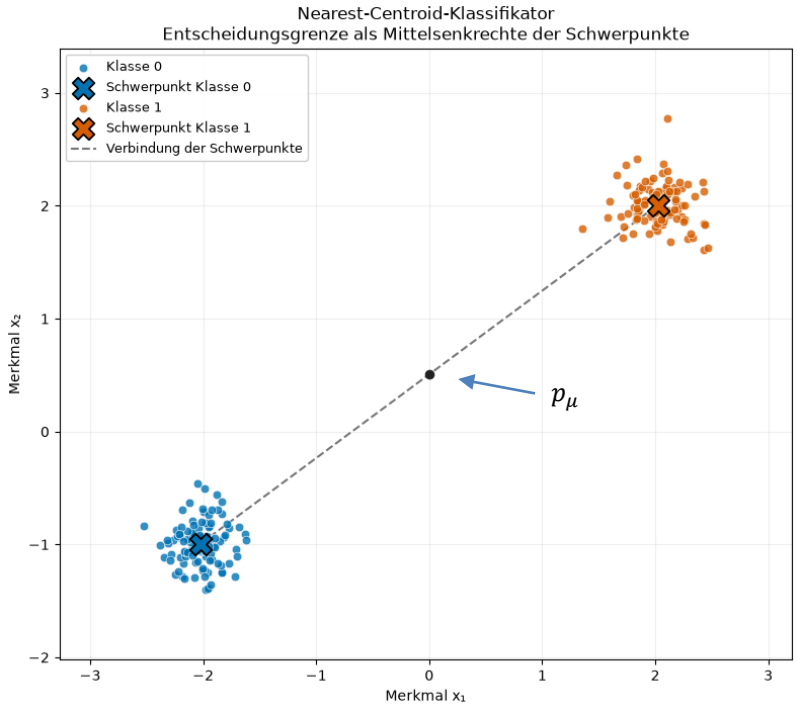
$$p_\mu = \left(\frac{\mu_{-,1} + \mu_{+,1}}{2}, \frac{\mu_{-,2} + \mu_{+,2}}{2}\right)$$

Steigungsverhältnis rechtwinkliger Geraden:

$$m = -\frac{1}{m_h}$$

Daraus folgt (Punkt-Steigungs-Form):

$$x_2 - p_{\mu,2} = -\frac{1}{m_h} \cdot (x_1 - p_{\mu,1})$$

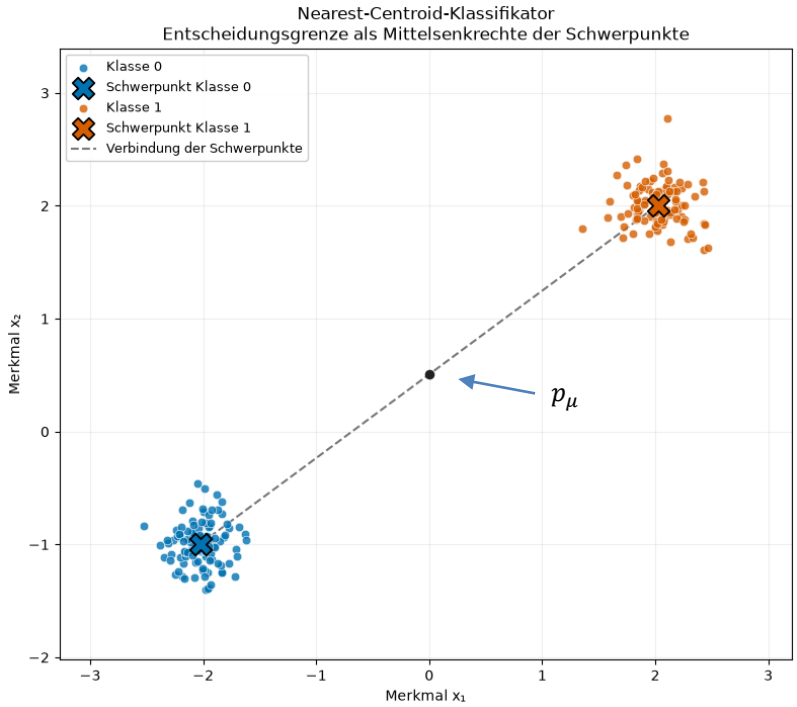


Eine handkonstruierte lineare Methode (Nearest-Centroid)

Idee:

- Schwerpunkte der Cluster über Mittelwerte berechnen
- Gerade zwischen den Schwerpunkten ziehen
- Rechtwinklige Gerade im Mittelpunkt ziehen

$$x_2 - p_{\mu,2} = -\frac{1}{m_h} \cdot (x_1 - p_{\mu,1})$$



Eine handkonstruierte lineare Methode (Nearest-Centroid)

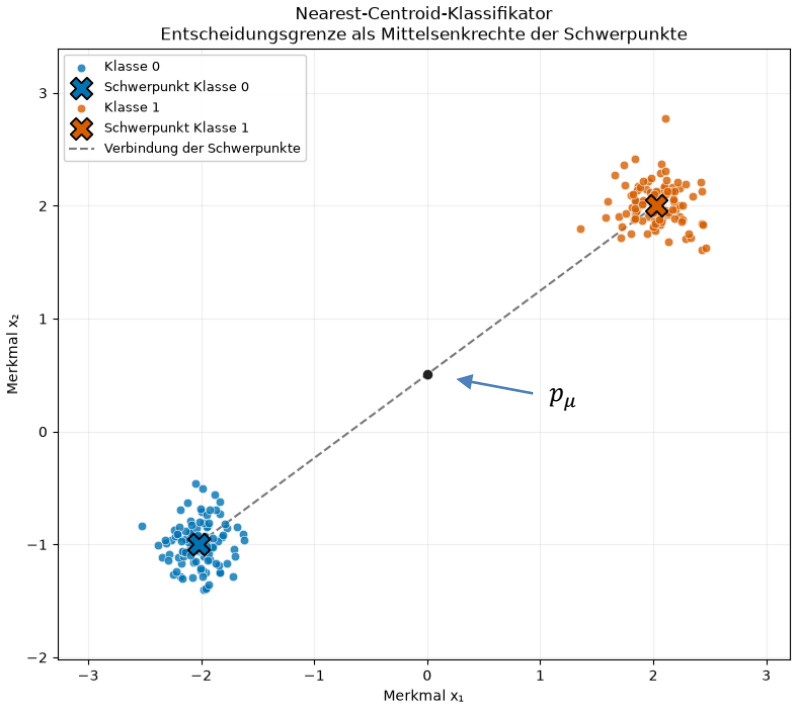
Idee:

- Schwerpunkte der Cluster über Mittelwerte berechnen
- Gerade zwischen den Schwerpunkten ziehen
- Rechtwinklige Gerade im Mittelpunkt ziehen

$$x_2 - p_{\mu,2} = -\frac{1}{m_h} \cdot (x_1 - p_{\mu,1})$$

$$m_h \cdot x_2 - m_h \cdot p_{\mu,2} = -x_1 + p_{\mu,1}$$

$$\underbrace{m_h}_{w_2} \cdot x_2 + \underbrace{1}_{w_1} \cdot x_1 \underbrace{- m_h \cdot p_{\mu,2} - p_{\mu,1}}_{b} = 0$$



Eine handkonstruierte lineare Methode (Nearest-Centroid)

Idee:

- Schwerpunkte der Cluster über Mittelwerte berechnen
- Gerade zwischen den Schwerpunkten ziehen
- Rechtwinklige Gerade im Mittelpunkt ziehen

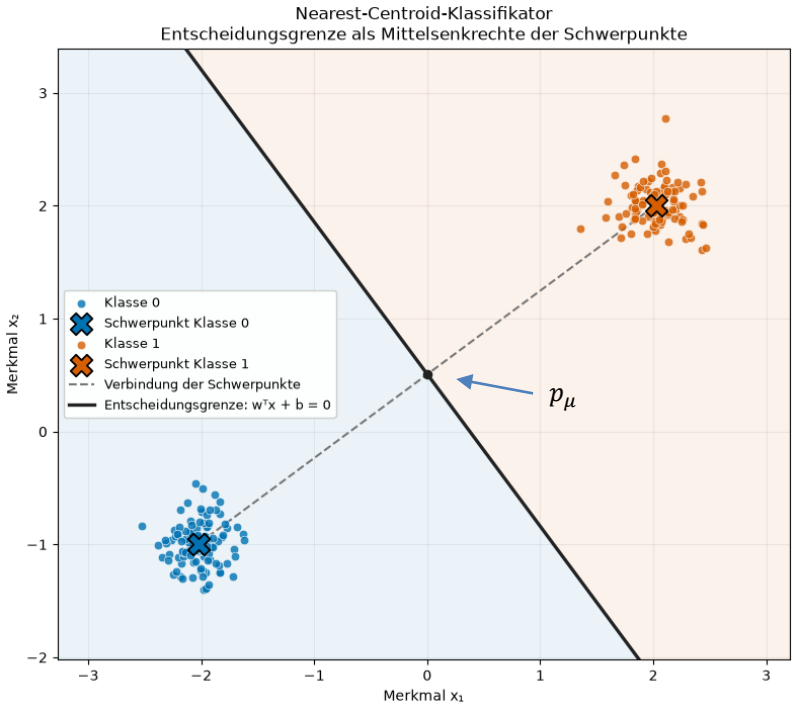
$$x_2 - p_{\mu,2} = -\frac{1}{m_h} \cdot (x_1 - p_{\mu,1})$$

$$m_h \cdot x_2 - m_h \cdot p_{\mu,2} = -x_1 + p_{\mu,1}$$

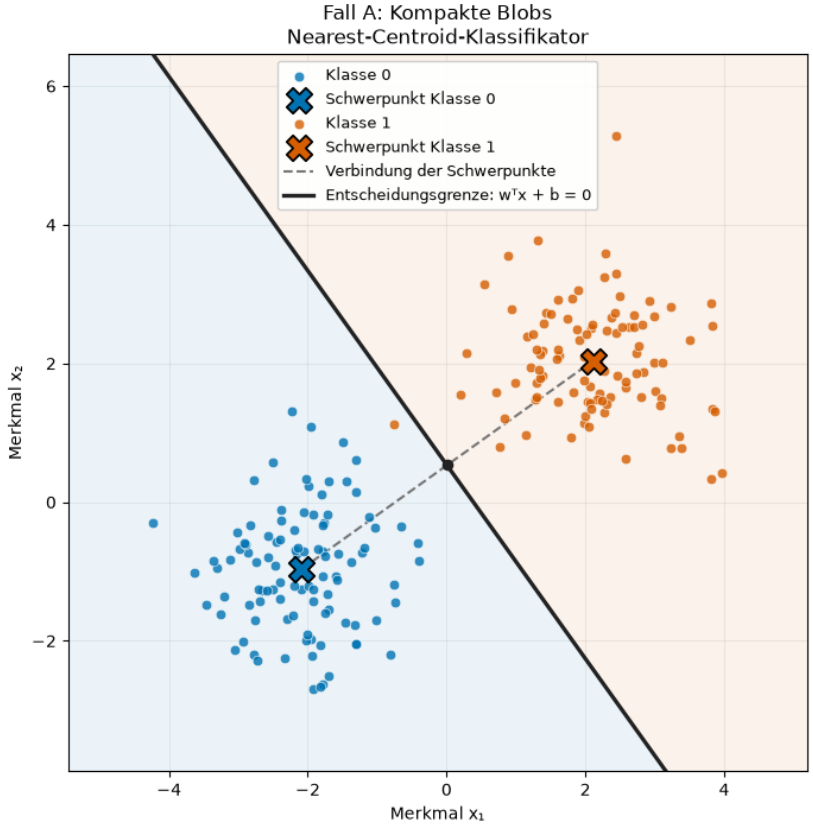
$$\underbrace{m_h}_{w_2} \cdot x_2 + \underbrace{1}_{w_1} \cdot x_1 \underbrace{- m_h \cdot p_{\mu,2} - p_{\mu,1}}_{b} = 0$$

$$g(x_1, x_2) = (m_h, 1) \begin{pmatrix} x_2 \\ x_1 \end{pmatrix} + b = 0$$

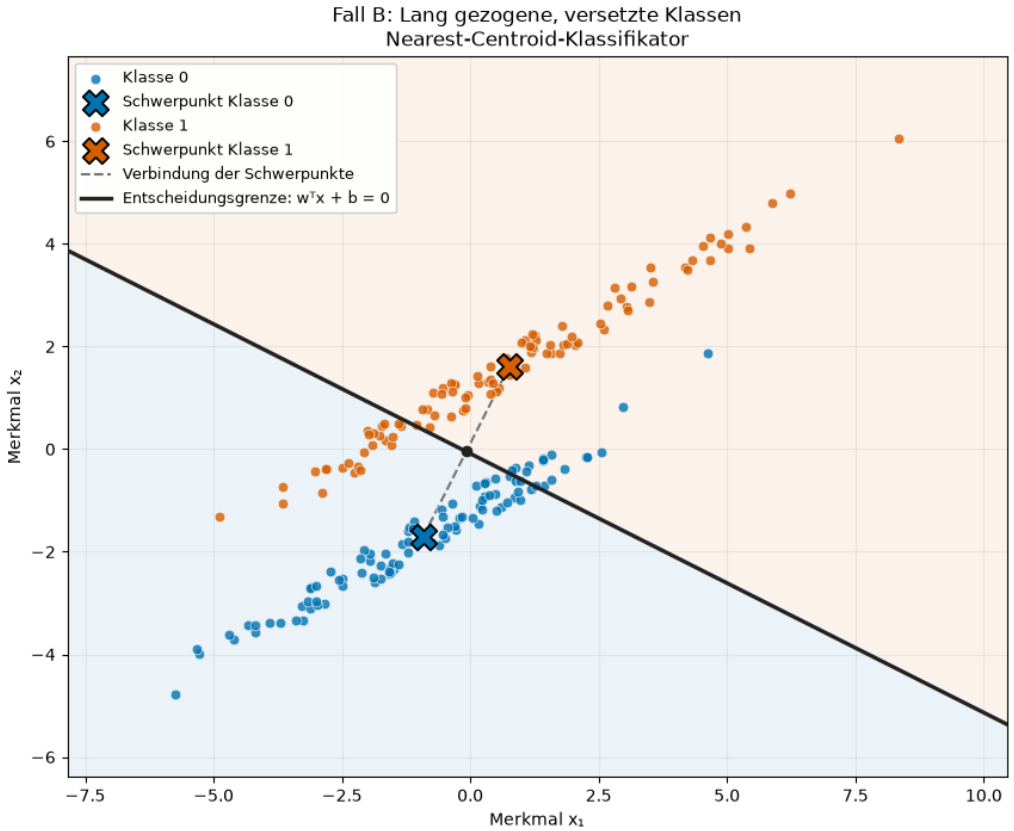
$$g(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x} + b = 0$$



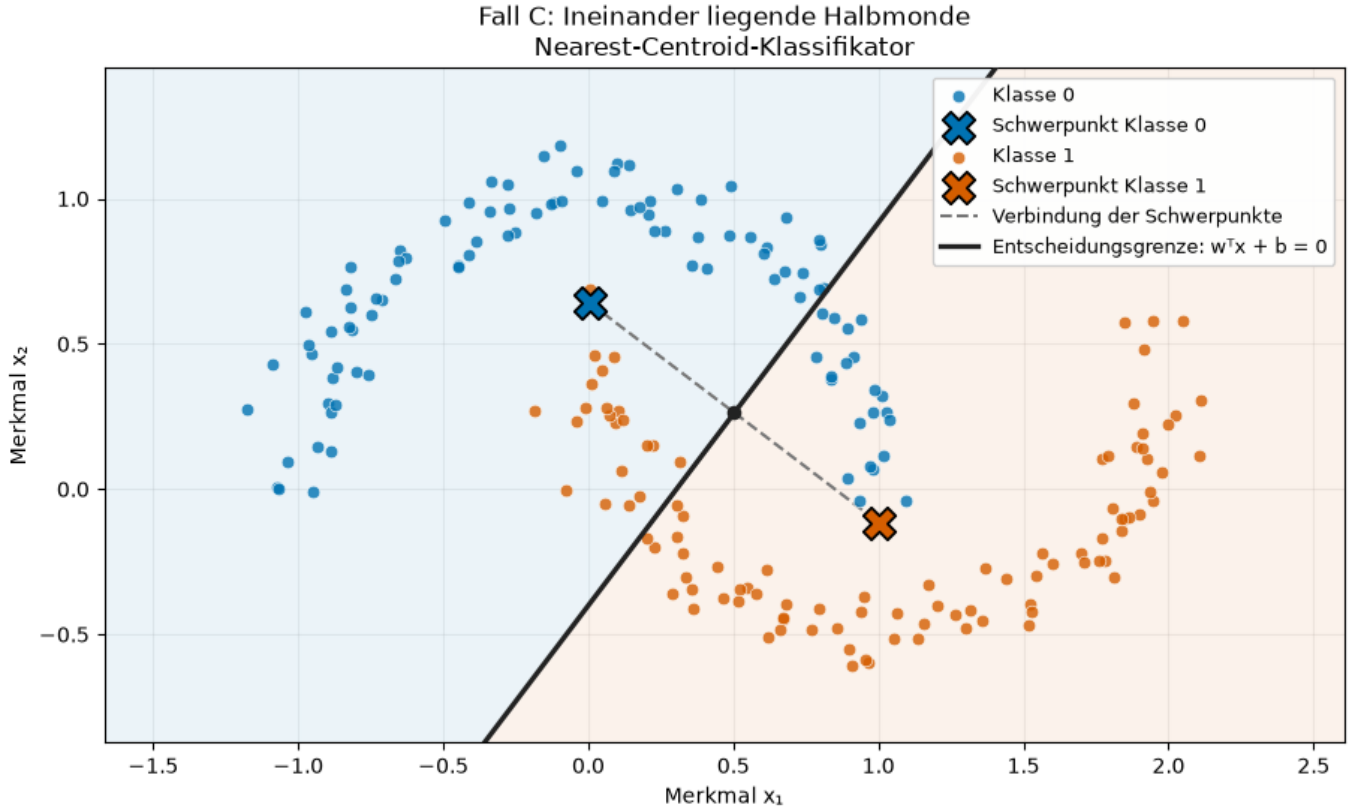
Eine handkonstruierte lineare Maschine (Nearest-Centroid)



Eine handkonstruierte lineare Maschine (Nearest-Centroid)



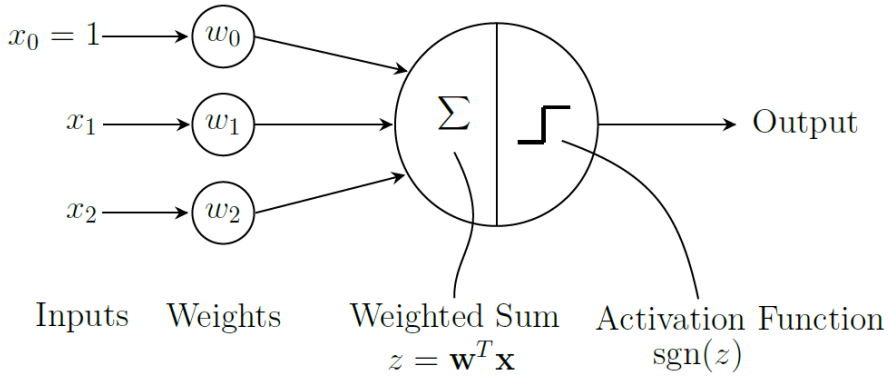
Eine handkonstruierte lineare Maschine (Nearest-Centroid)



Das einfache Perzeptron

Das einfache Perzeptron

Einfaches Perzeptron: Linearer Klassifizierer mittels eines einzigen technischen Neurons



Inputs: $x_1, \dots, x_n$

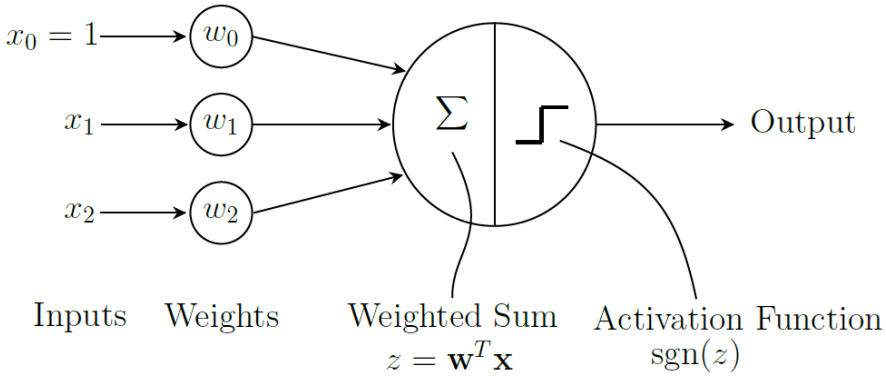
Gewichte: $w_0, \dots, w_n$, wobei w_0 dem Bias b entspricht

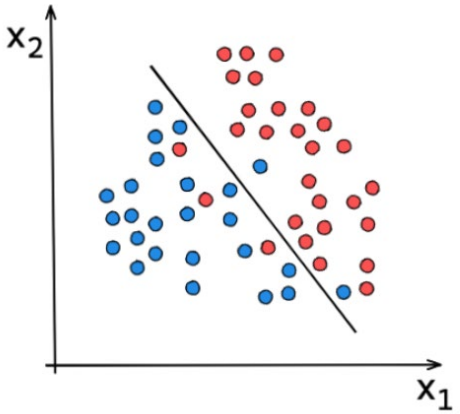
Gewichtete Summe: $z = w_0 \cdot x_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_n \cdot x_n = \boldsymbol{w}^T \cdot \boldsymbol{x}$

Aktivierungsfunktion: $h(z) = \mathrm{sgn}(\boldsymbol{w}^T \cdot \boldsymbol{x})$

Ausgabe: $\hat{y} = h(z)$

Das einfache Perzeptron





Dimension:

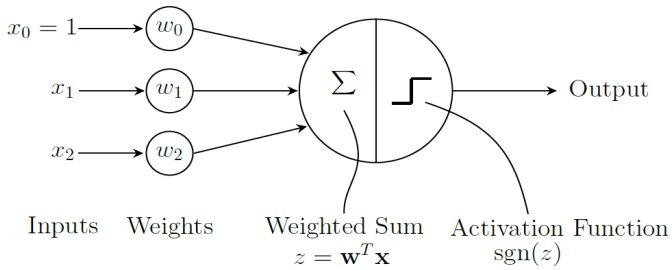
$d = 2$: Gerade

$d = 3$: Ebene

$d > 3$: Hyperebene

Das einfache Perzeptron

<u>Wie lernt ein einfaches Perzeptron?</u>



Klassenlabel: $y_i = \{-1, 1\}$

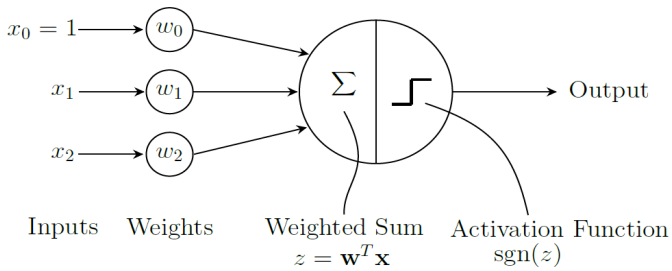
Für jeden Datenpunkt im Trainingsdatensatz (iterativ):

Wenn Klassifikation korrekt ($\hat{y}_i = y_i$):

kein Update

Das einfache Perzeptron

Wie lernt ein einfaches Perzeptron?



Klassenlabel: $y_i = \{-1, 1\}$

Für jeden Datenpunkt im Trainingsdatensatz (iterativ):

Wenn Klassifikation korrekt ($\hat{y}_i = y_i$):

kein Update

Wenn Klassifikation inkorrekt ($\hat{y}_i \neq y_i$):

Vorzeichenmodifikation

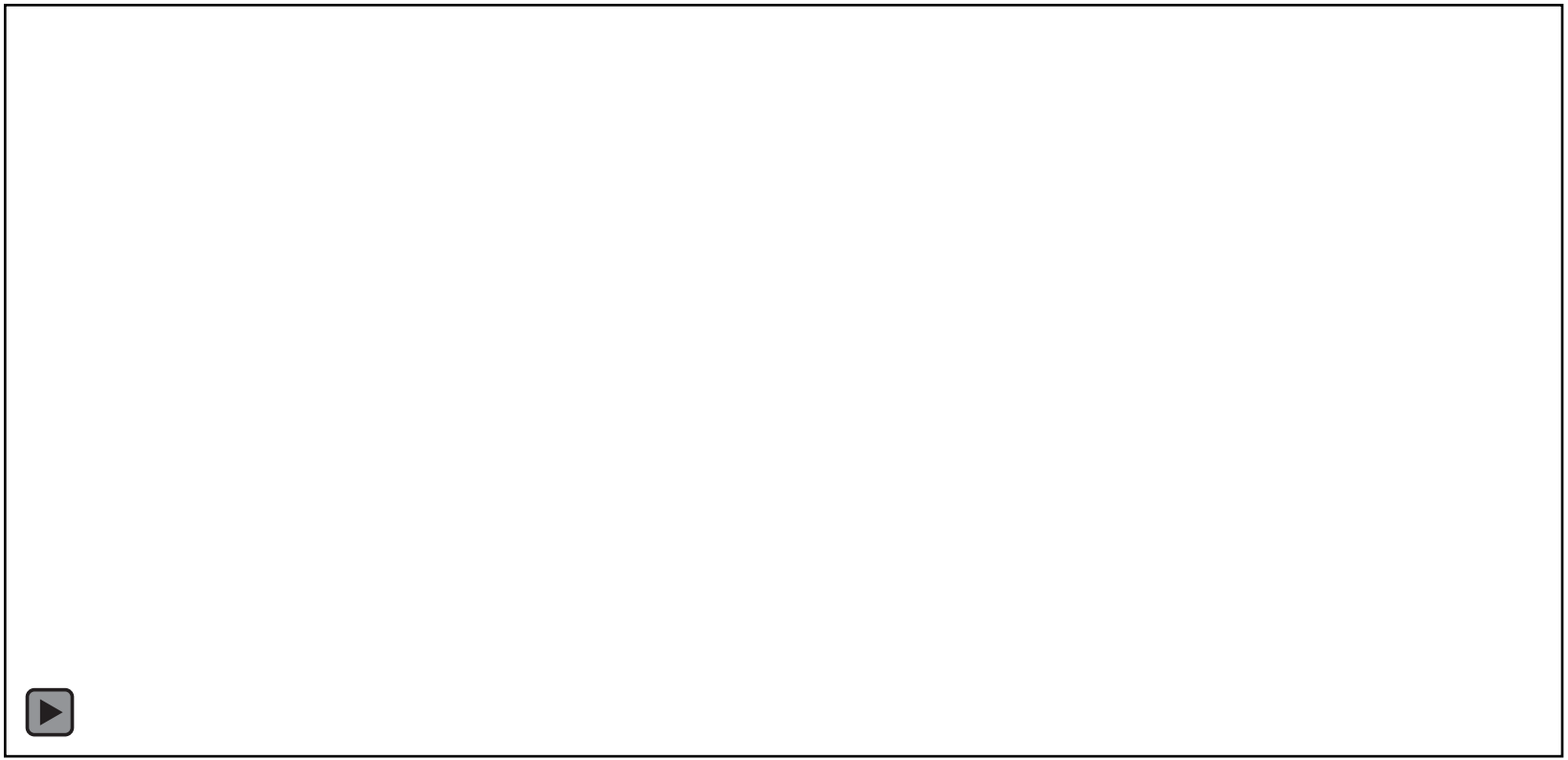
$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta \cdot y_i \cdot \boldsymbol{x}_i$$

Nicht vergessen:
$w_0 = b$ und $x_0 = 1$

$$\eta \in \mathbb{R}_+$$

Lernrate
(Hyperparameter)

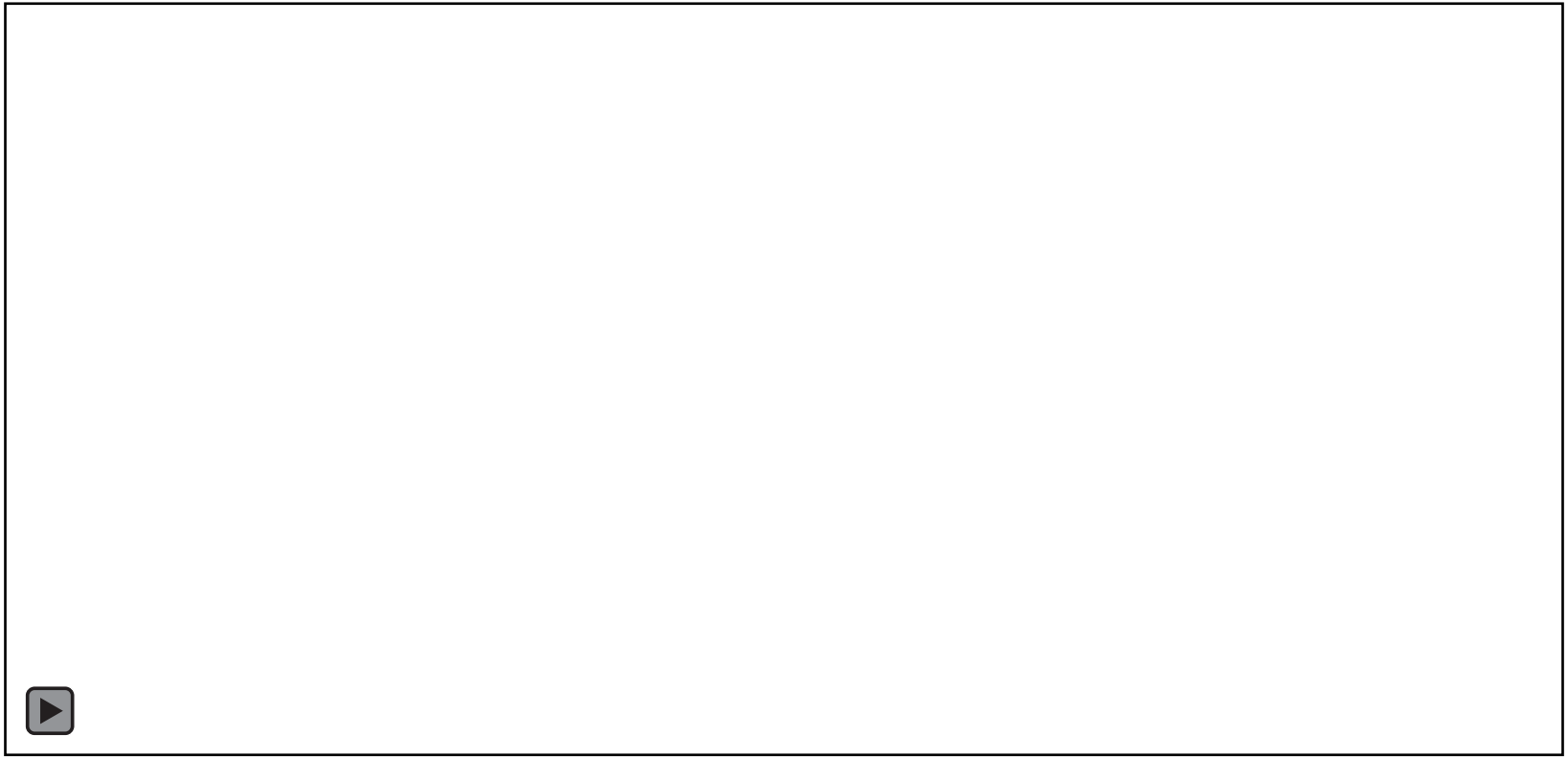
Das einfache Perzeptron



Das einfache Perzeptron



Das einfache Perzeptron



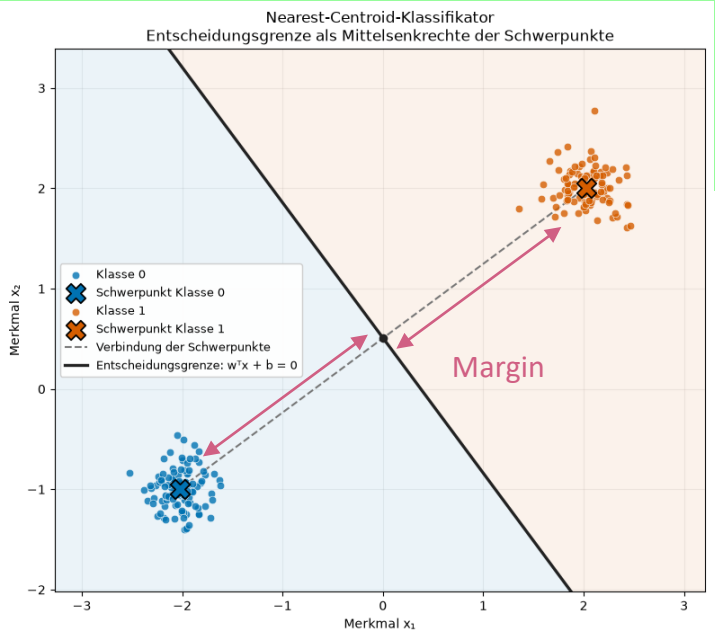
Konvergenzsatz für das Perzeptron

Bewiesen durch Novikoff[1]

Ein Perzeptron konvergiert für einen linear separierbaren Datensatz D nach höchstens

$$\frac{R}{\gamma^2} \qquad (\mathrm{R} = \max_{x,y \in D} ||x||_2, \gamma = \text{Margin})$$

Updateschritten bzw. Fehlern (unabhängig von der Lernrate).

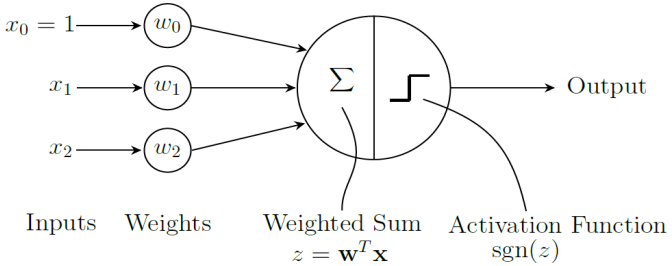


Bedeutung 1: Je größer der Margin, desto leichter ist das Problem für das Perzeptron.

Bedeutung 2: Je größer *R* ist (weil Eingangsdaten beispielsweise extrem weit vom Ursprung entfernt liegen), desto größer ist die maximale Anzahl an Updateschritten.
→ Deswegen: Normierung / Skalierung wichtig!

[1]Novikoff, Albert BJ. "On convergence proofs on perceptrons." *Proceedings of the Symposium on the Mathematical Theory of Automata*. Vol. 12. No. 1. 1962.

Können wir eine Regel definieren, die nicht nur zwischen richtig und falsch unterscheidet, sondern auch beschreibt, wie schlecht unsere aktuellen Modellparameter sind?

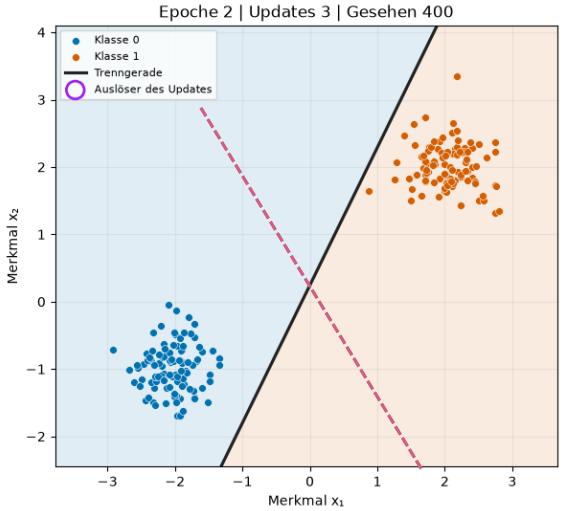


Nachteile der bisherigen Aktualisierungsregel

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta \cdot y_i \cdot \boldsymbol{x}_i$$

$$\eta \in \mathbb{R}_+$$

1. **Keine gezielte Optimierung der Entscheidungsgrenze**
   - Bei fehlerfreier Klassifikation erfolgen keine weiteren Anpassungen.
   - Die gefundene Entscheidungsgrenze besitzt nicht zwangsläufig einen optimalen Margin.
2. **Keine fehlerabhängig abgestufte Gewichtsanpassung**
   - Die Aktualisierung berücksichtigt nicht, wie *stark* (Entfernung zur aktuellen Trenngerade) ein Datenpunkt falsch klassifiziert wird.
   - Keine Unterscheidung zwischen Rotation und Verschiebung der Trenngeraden (es wird nicht auf den „individuellen Beitrag“ jedes einzelnen Gewichts auf den Fehler geschaut).

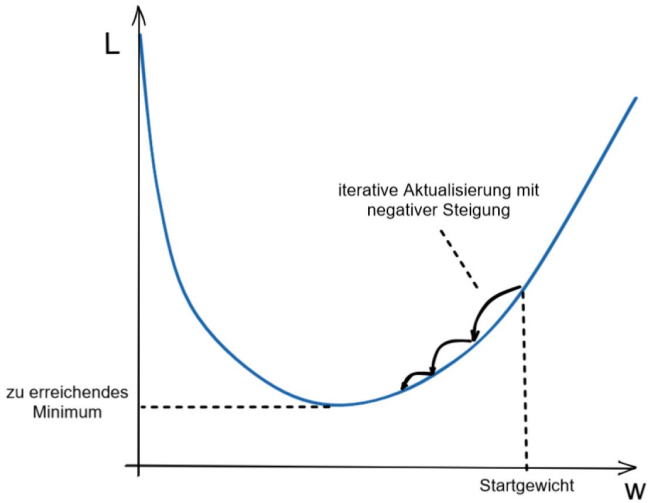


Gradienten-basiertes Lernen

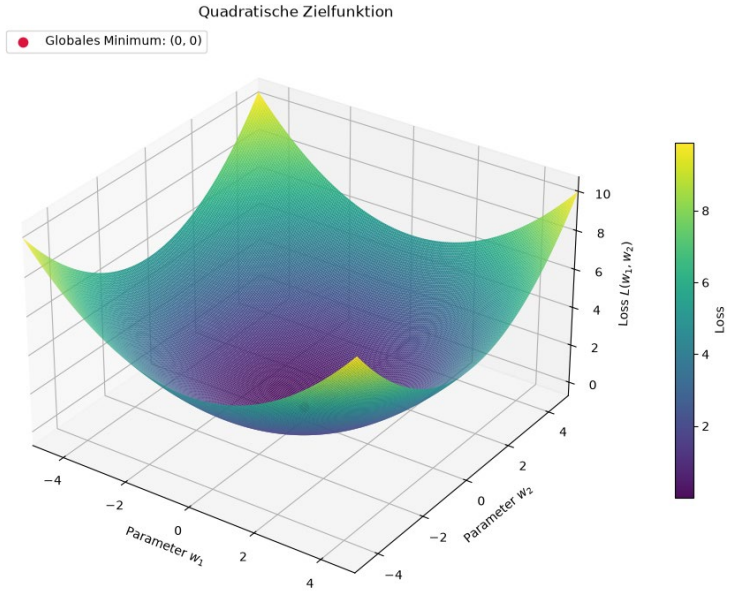
Wir definieren eine Fehler-, Loss- bzw. Kostenfunktion, die misst, wie schlecht unsere aktuellen Parameter sind:

$$L(\boldsymbol{w}) - \text{z.B. } L(\boldsymbol{w}) = (y - \hat{y})^2.$$

Die Parameter w werden schrittweise so angepasst, dass L minimiert wird.



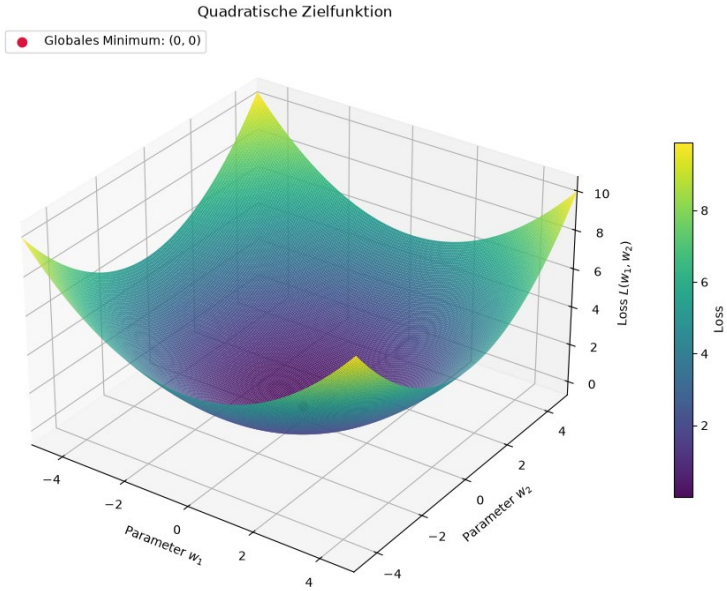
1-dimensionales Lernproblem



2-dimensionales Lernproblem

Die Verlustlandschaft (*loss landscape*)

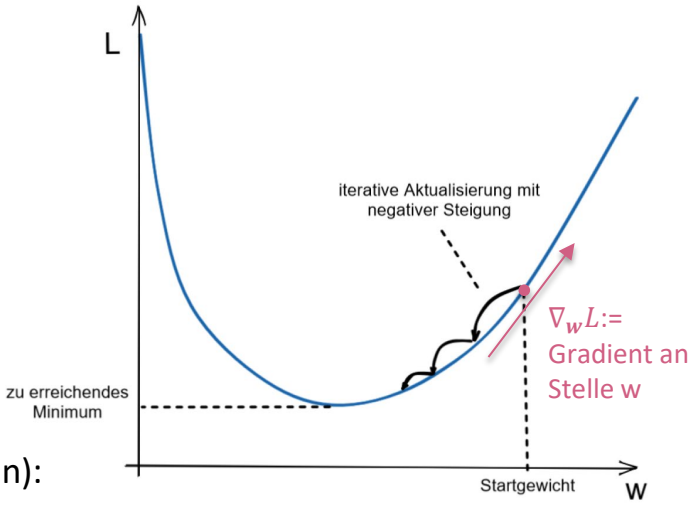
- Die Ausgestaltung der Landschaft hängt ab von
  - den Daten,
  - dem Hypothesenraum (Modellarchitektur),
  - der Aktivierungsfunktion und
  - der Loss-Funktion.
- **Lineare Klassifikationsmodelle** führen bei Wahl einer konvexen Loss-Funktion immer zu einem konvexen Optimierungsproblem
  → jedes lokale Minimum ist ein globales Minimum.
- Bei nicht-linearen Modellen(z. B. tiefen neuronalen Netzen) und Problemen wird die Landschaft hochgradig nicht-konvex. Typische Phänomene sind:
  - lokale Minima,
  - Sattelpunkte.



Gradient Descent Aktualisierungsregel

$$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta \cdot \nabla_{\boldsymbol{w}} L$$

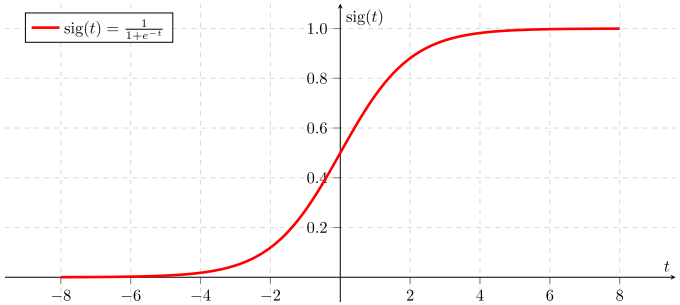
$$w_i \leftarrow w_i - \eta \cdot \frac{\delta L}{\delta w_i}$$

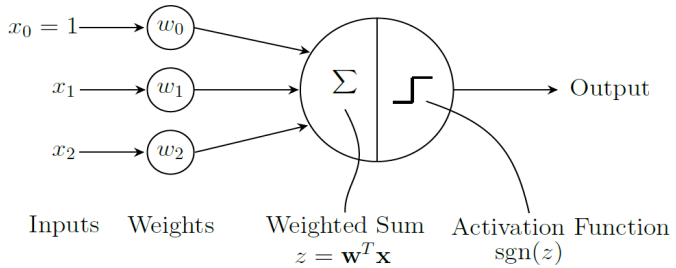


Problem: Signum-Funktion ist nicht differenzierbar!

Lösung: Aktivierungsfunktion austauschen (z.B. Sigmoid-Funktion):

$$h(z) = \frac{1}{1+e^{-z}}$$





Gradient berechnen durch Kettenregel:

$$\frac{\delta L}{\delta w_i} = \frac{\delta L}{\delta h} \cdot \frac{\delta h}{\delta z} \cdot \frac{\delta z}{\delta w_i}$$

Beispiel:

$L(\boldsymbol{w}) = (y - \hat{y})^2$

$h(z) = \text{sig}(z) = \frac{1}{1+e^{-z}}$

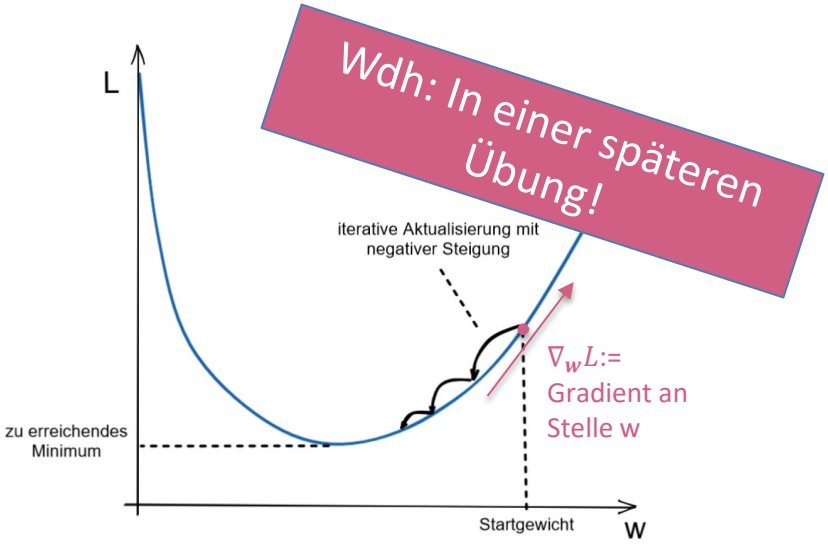
$z = w_0 \cdot x_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_n \cdot x_n = \boldsymbol{w}^T \cdot \boldsymbol{x}$

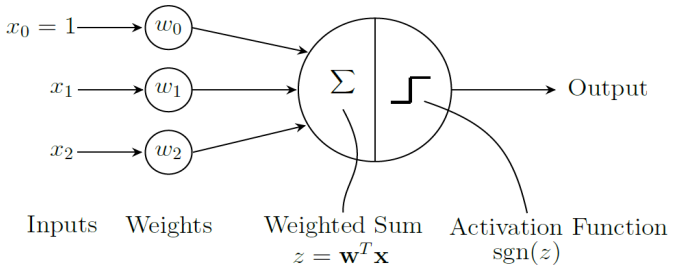
Aktualisierung

$$\frac{\delta L}{\delta h} = 2 \cdot (y - \hat{y}) \cdot (-1)$$

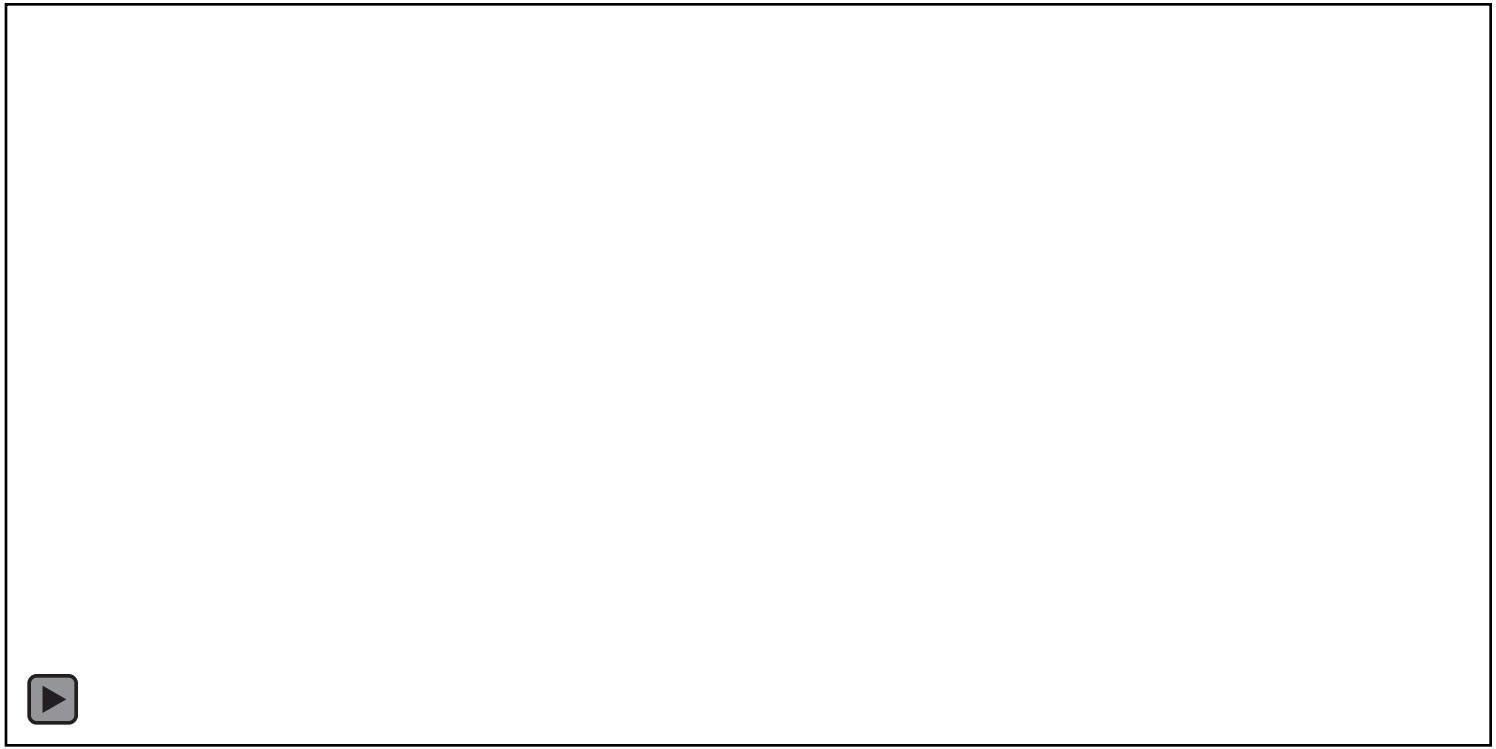
$$\frac{\delta h}{\delta z} = \text{sig}(z)(1 - \text{sig}(z))$$

$$\frac{\delta z}{\delta w_i} = x_i$$



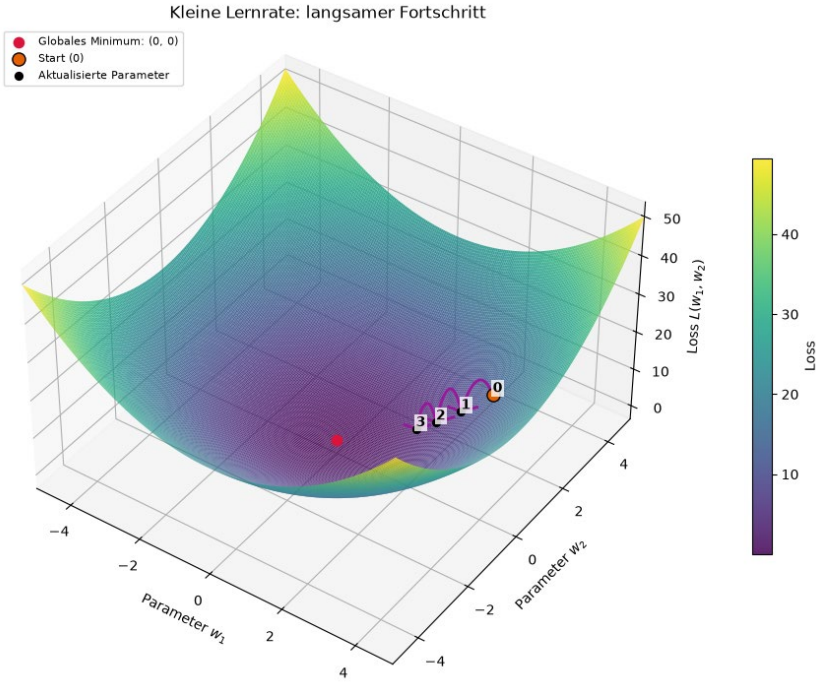


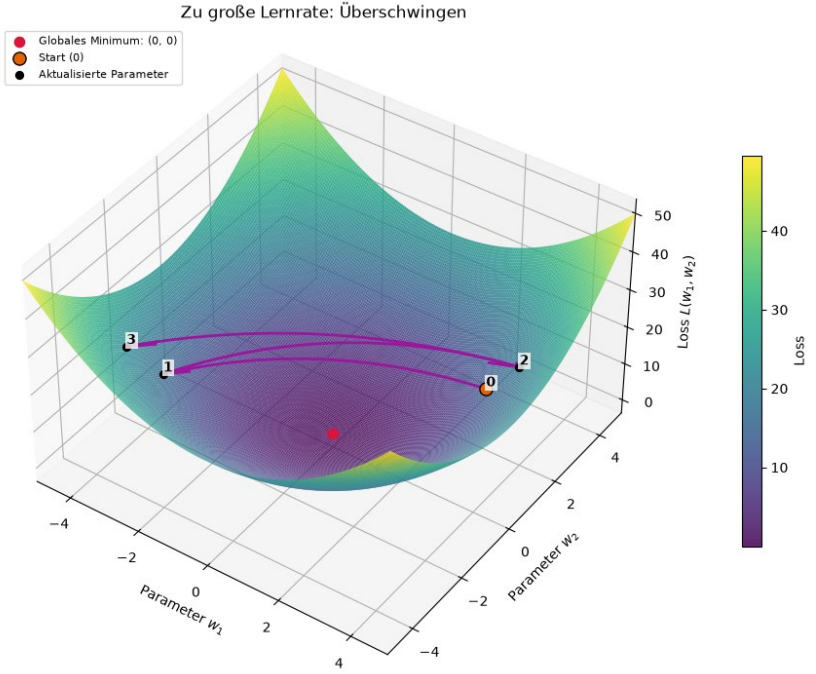
Gradienten-basiertes Lernen



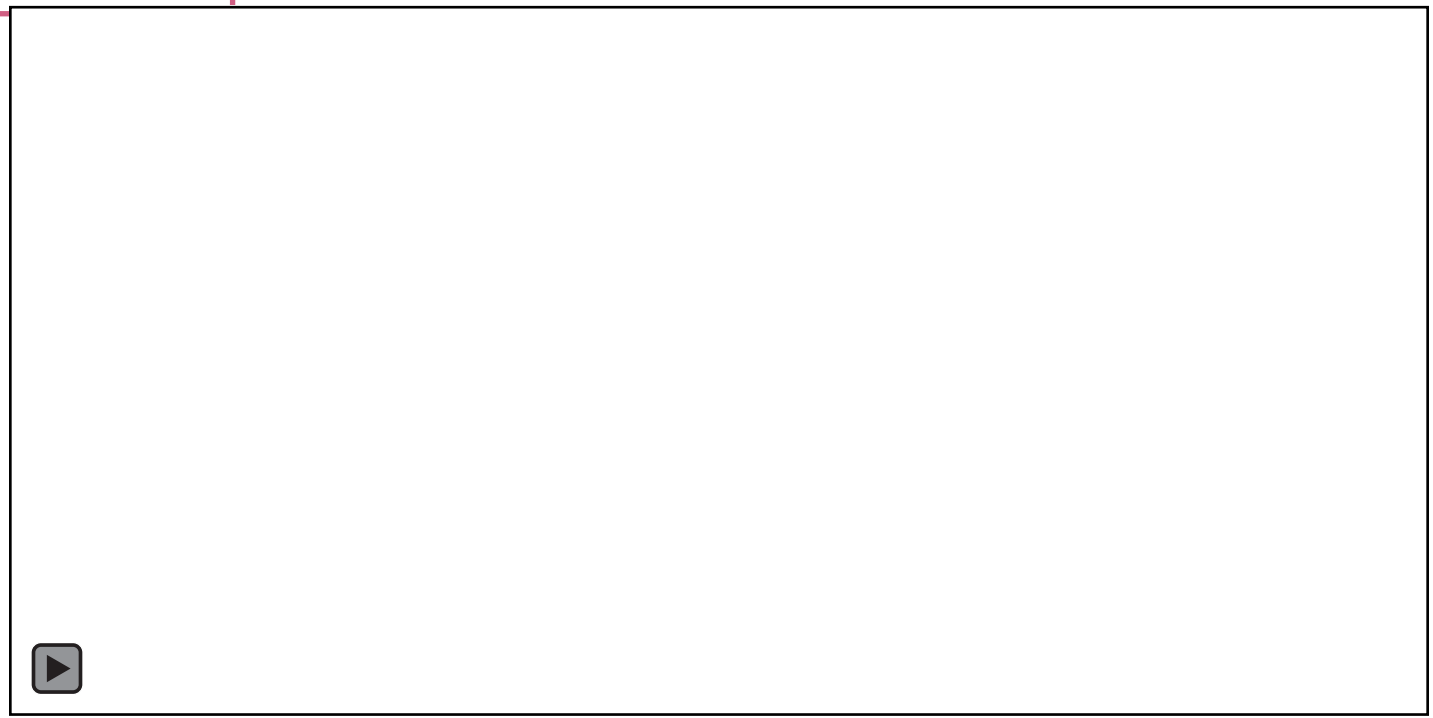
Lernrate

Zu große Lernrate
Fehlerbild 1
Oszillation („Jojo-Effekt“)

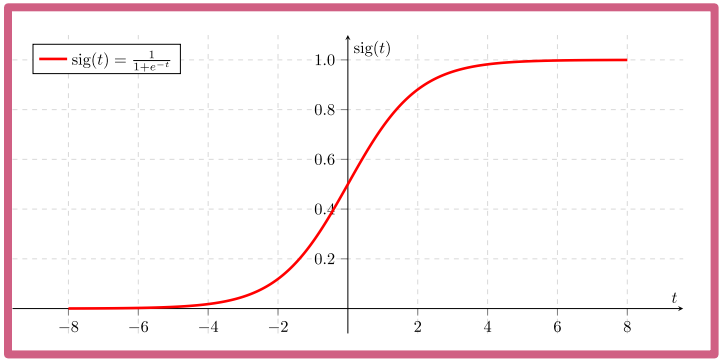




Zu große Lernrate
Fehlerbild 2



Zu große Lernrate
Fehlerbild 2



w = [36.336, 94.686]

$$h(z) = \mathrm{sig}(z) = \frac{1}{1 + e^{-z}}$$

Wenn $z \gg 0$ oder $z \ll 0$:

$$\frac{\delta h}{\delta z} \approx 0$$

Bedeutet:

$$\frac{\delta L}{\delta w_i} \approx 0$$

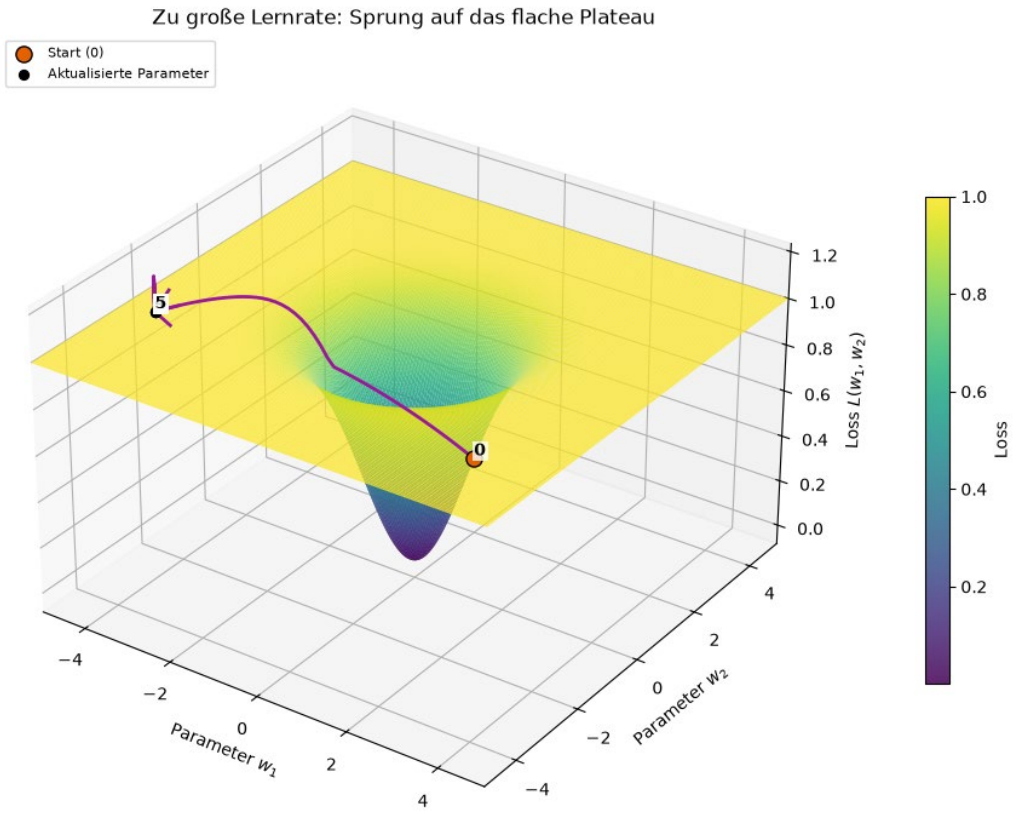
Gradienten-basiertes Lernen

Zu große Lernrate
Fehlerbild 2

Die „Flachland-Falle“

Zu große Lernrate
Fehlerbild 3

Numerische Instabilität





Arbeitsgruppe Diskrete Systeme
Dr. Christoph-Alexander Holst
Institut für industrielle Informationstechnik (iniT)
Campusallee 6
32657 Lemgo

